Facultatea de Automatică şi Calculatoare, UPB
Departamentul de Calculatoare

Analiza algoritmilor

ŞL.dr.ing. Andrei Mogoş

2020-2021

- Calculabilitate (2) -

Obs:

S-a demonstrat că mulțimea funcțiilor recursive este identică cu mulțimea funcțiilor Turing-calculabile.

Obs:

Considerând adevărata Teza Church-Turing rezultă că o funcție $f: \mathbb{N} \to \mathbb{N}$ este recursivă $\Leftrightarrow$ este efectiv calculabilă.

Notație

Notăm cu $F_{\mathcal{R}}$ mulțimea funcțiilor recursive din $Hom(\mathbb{N}, \mathbb{N})$.

Prop 5

Mulțimea $F_{\mathcal{R}}$ este ∞ - numărabilă.

Dem

Construim o numerotare pentru $F_{\mathcal{R}}$:

f_0 = funcția calculată de programul P_0

f_j = funcția calculată de programul cu indicele cel mai mic din mulțimea:

$P_{1,1} \setminus \{P \in P_{1,1} / P \text{ calculează } f_k, k < j\}$

pentru $j > 0$.

$\Rightarrow F_r$ este numărabilă. (1)

• Arătăm că F_R este infinită. Pp. abs că F_R este finită și că are forma:

$$F_R = \{f_1, f_2, \dots, f_m\}, \quad m \in \mathbb{N}$$

Construim funcția:

$$f^*: \mathbb{N} \to \mathbb{N},$$

$$f^*(n) = \begin{cases} \max\{f(0) \mid f \in F_R\} + 1, & n = 0 \\ f_1(n), & n > 0 \end{cases}$$

$f^* \notin F_R$, dar este recursivă $\Rightarrow$ Abs.

Prin urmare, $F_{\mathbb{R}}$ este infinită. (2)

- Din (1) și (2) rezultă că mulțimea $F_{\mathbb{R}}$ este ∞ - numărabilă.

Prop 6 :

$\mathrm{Hom}(\mathbb{N},\mathbb{N}) \setminus F_r \neq \emptyset$.

Dem

$\mathrm{Hom}(\mathbb{N},\mathbb{N})$ ∞ - nenumărabilă

F_r ∞ - numărabilă

$F_r \subseteq \mathrm{Hom}(\mathbb{N},\mathbb{N})$

| Prop 2 $\Longrightarrow$

$\Longrightarrow \mathrm{Hom}(\mathbb{N},\mathbb{N}) \setminus F_r \neq \emptyset$.

Obs:

În Prop 6 observăm că $\mathrm{Hom}(\mathbb{N}, \mathbb{N})$ este mulțimea problemelor de la $\mathbb{N}$ la $\mathbb{N}$, iar $\mathcal{F}_R$ este mulțimea problemelor de la $\mathbb{N}$ la $\mathbb{N}$ rezolvabile cu calculatoarele din ziua de astăzi.

Prop 6 spune că există probleme de la $\mathbb{N}$ la $\mathbb{N}$ care nu se pot rezolva folosind calculatoarele din ziua de astăzi.

Obs:

Ținând cont de Prop 6 avem:

- dacă Teza Church-Turing este adevărată atunci întotdeauna vor exista probleme de la IN la IN nerezolvabile automat;
- dacă Teza Church-Turing este falsă atunci este posibil ca în viitor orice problemă de la IN la IN să poată fi rezolvată automat.

Def: O mulțime $A \subseteq \mathbb{N}$ s.n. **recursivă** dacă există o funcție **recursivă** $f: \mathbb{N} \to \mathbb{N}$ astfel încât:

$$f(n) = \begin{cases} 1, & n \in A \\ 0, & n \notin A \end{cases}, \quad \forall n \in \mathbb{N}.$$

Obs:

O mulțime recursivă $A \subseteq \mathbb{N}$ are proprietatea că se poate decide în mod automat (folosind un program din $P_{1,1}$) dacă o valoare oarecare $n \in \mathbb{N}$ aparține mulțimii A.

Def (*): O mulțime $A \subseteq \mathbb{N}$ s.n. recursiv - numărabilă dacă există o funcție recursivă $f : A \to \mathbb{N}$.

Def (**): O mulțime $A \subseteq \mathbb{N}$ s.n. recursiv - numărabilă dacă există un program $Q \in P_{0,1}$ care la fiecare apel să calculeze un element din A. Q s.n. generator al lui A.

Prop 7:

Definițiile (*) și (**) sunt echivalente.

Dem

Fie $A \subseteq \mathbb{N}$. Arătăm că:

$\exists f : A \to \mathbb{N}$ recursivă $\Leftrightarrow \exists Q \in P_{0,1}$ generator al lui A.

"$\Rightarrow$": Fie $f : A \to \mathbb{N}$ o funcție recursivă.

Rezultă că există un program $P \in P_{1,1}$ astfel încât $P(n) = \begin{cases} f(n), & n \in A \\ \perp, & n \notin A \end{cases}$

Construim un generator al lui A:

```
Q ( )
    static A = ∅ , n = 0
    while (1)
        for (m = 0, m ≤ n, m++)
            if (P(m) se termină în n unități de timp
                și m ∉ A )
                A = A ∪ {m}
                return m
        n++
```

Obs: Se consideră că variabilele statice își păstrează valorile între apelurile programului.

Obs: Se observă că programul Q este un generator al lui A: P(m) se termină dacă pentru valoarea m ∈ A și Q generează în timp finit la fiecare apel un element al lui A.

"<=": Fie $Q \in P_{0,1}$ un generator al lui A.

Construim programul:

```
P(n)
    static A = ∅
    while (1)
        if (n ∈ A)
            return 1
        A = A ∪ {Q()}
```

Programul P calculează funcția recursivă $f: A \to \mathbb{N}$

$f(x) = 1, \forall x \in A$

$\Rightarrow \exists f: A \to \mathbb{N}$ recursivă

Prop 8:

O mulțime recursivă este recursiv-numărabilă.

Dem

Fie $A \subseteq \mathbb{N}$ o mulțime recursivă. Rezultă că există o funcție recursivă $f: \mathbb{N} \to \mathbb{N}$ și un program $P \in P_{1,1}$ care calculează funcția f astfel încât $A = \{ n \in \mathbb{N} \mid P(n) = 1 \}$.

Construim un generator al lui A:

```
Q ( )
    static A = ∅
    for ( m = 0, , m++)
        if ( P(m) = 1 și m ∉ A)
            A = A ∪ {m}
            return m
```

$\Rightarrow$ A este <u>recursiv - numărabilă</u>.

Prop 9:

Există mulțimi:

a) recursive;

b) care sunt recursiv - numărabile, dar nu sunt recursive;

c) care nu sunt recursive și nici recursiv - numărabile.

Def: Fie $T : \mathbb{N} \to \{0,1\}$ un predicat și $M_T = \{n \in \mathbb{N} \mid T(n) = 1\}$ mulțimea de adevăr a lui T.

- T s.n. **decidabil** dacă M_T este recursivă;
- T s.n. **semidecidabil** dacă M_T este recursiv - numărabilă, dar nu este recursivă;
- T s.n. **nedecidabil** dacă M_T nu este nici recursivă și nici recursiv - numărabilă.

Ex: Problemă decidabilă (predicat decidabil)

Fie $n \in \mathbb{N}$. Să se stabilească dacă n este număr prim.

Explicație: se poate construi un algoritm determinist care să răspundă în timp finit dacă n este număr prim sau nu. Prin urmare: și în caz afirmativ și în caz negativ răspunsul se obține în timp finit.

Ex: Problemă semidecidabilă (predicat semidecidabil)

Fie A un program și x o intrare a lui A. Calculul A(x) se termină? (Problema terminării programelor).

Explicație

```
B(A, x)
    if A(x) se termină
        return 1
    return 0
```

- B(A, x) întoarce 1 dacă A(x) se termină;
- B(A, x) nu se termină dacă A(x) nu se termină.
- Se obține rezultat în timp finit doar pe caz afirmativ.

Ex: Problemă nedecidabilă (predicat nedecidabil)

Fie A un program și x o intrare a lui A. Calculul A(x) nu se termină? (Problema "neterminării" programelor).

Explicație

```
B(A, x)
    if A(x) nu se termină
        return 1
    return 0
```

- Nu se obține rezultat în timp finit pe caz afirmativ.

Obs:

- Problemele decidabile pot fi rezolvate automat (folosind calculatoarele din ziua de astăzi);
- Problemele semidecidabile și nedecidabile nu pot fi rezolvate automat (folosind calculatoarele din ziua de astăzi).